

Sprog og oversættelse

Lexical analysis: From NFA to DFA; Minimizing a DFA

Part 3: Building a lexer

Hans Hüttel

(using content by Cosmin Oancea)

Creative Commons License
Attribution Non Commercial 4.0 International
(CC-BY-NC-4.0)

Back to lexical analysis

We can use a DFA to find an answer to the problem

Is an input w described by regular expression α ?

But how should we recognize a whole sequence of characters such as the program

```
// My program\n val result =\n   let val x = 10 :: 20 :: 0x30 :: []\n   in\n     List.map (fn a => 2 * 2 * a) x\n   end
```

and produce a sequence of tokens:

```
Keywd_Val, Id "result", Equal, Keywd_Let, Keywd_Val, Id "x", Equal, Int 10,\nOp_Cons, Int 20, Op_Cons, Int 48, Op_Cons, LBracket, RBracket, Keywd_in,\nId "List", Dot, Id "map", LParen, Keywd_fn Id "a", Arrow, Int 2, Multiply,\nInt 2, Multiply, Id "a", RParen, Id "x", Keywd_end
```

?

The strategy of a lexer

- Recognize prefixes of input as tokens,
- Restart the DFA on the input that remains after it has accepted part of the input
- But often, several decompositions of the input are possible. The lexer must be able to handle this.

Which token do we have?

Is

1298342

the numeral 1 followed by more numerals or one big numeral?

Is

function

the identifier func followed by the identifier tion or is this the keyword function?

Longest and First Match

Definition (Principle of longest match)

A lexical analyser usually outputs the token that consumes the longest part of the input

Longest and First Match

Definition (Principle of longest match)

A lexical analyser usually outputs the token that consumes the longest part of the input

This is important when reading identifiers and numbers! With this, 1298342 is a single token.

Tokens are usually prioritized, so the lexer can decide which token to recognize.

Definition (Principle of first match)

If two tokens are possible for the same input, choose the match with highest priority.

Longest and First Match

Definition (Principle of longest match)

A lexical analyser usually outputs the token that consumes the longest part of the input

This is important when reading identifiers and numbers! With this, 1298342 is a single token.

Tokens are usually prioritized, so the lexer can decide which token to recognize.

Definition (Principle of first match)

If two tokens are possible for the same input, choose the match with highest priority.

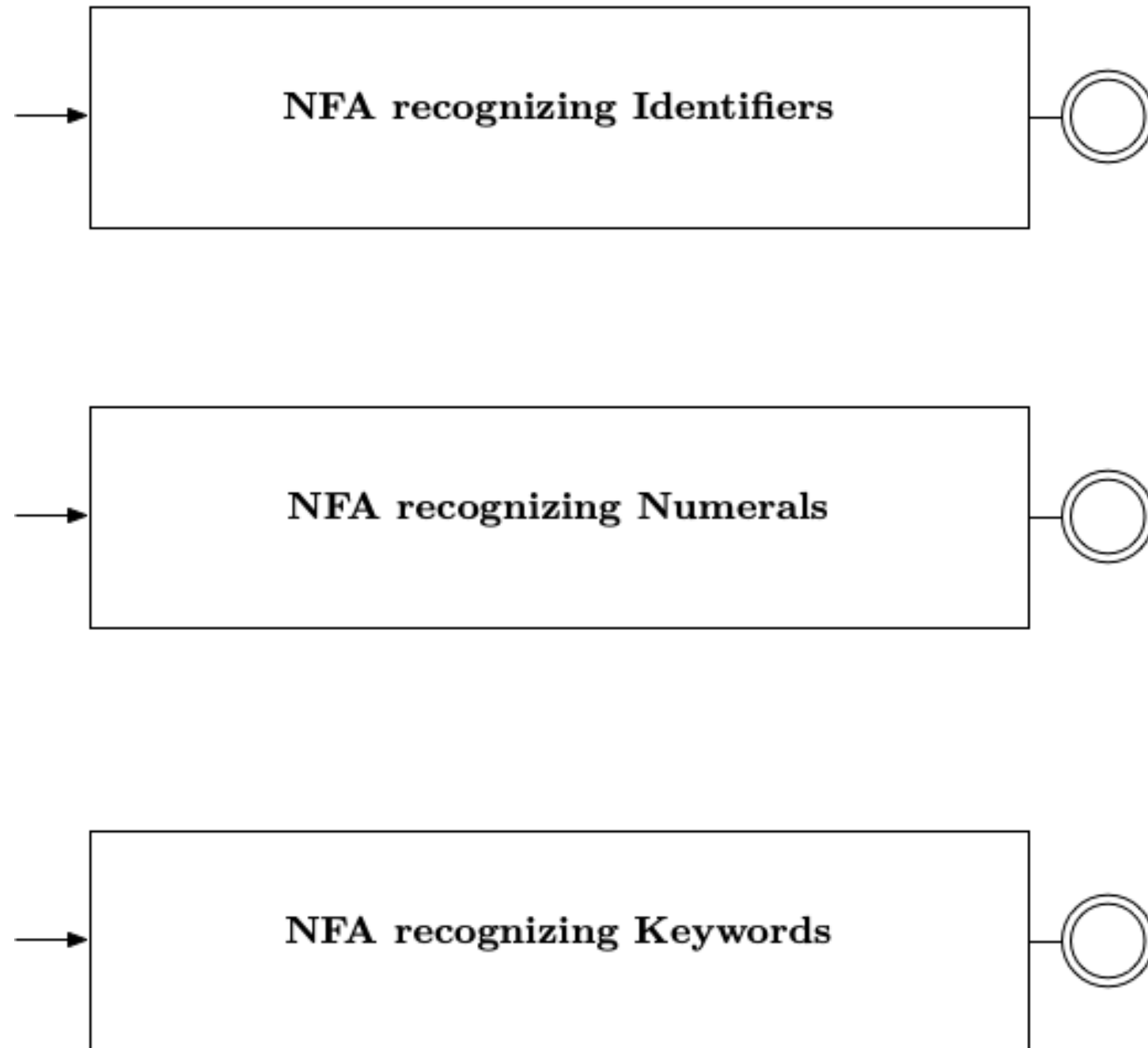
This is important when reading keywords and ensures that we see `function` as an identifier.

Building a lexer

A lexer is a combined DFA with prioritized final states and backtracking.

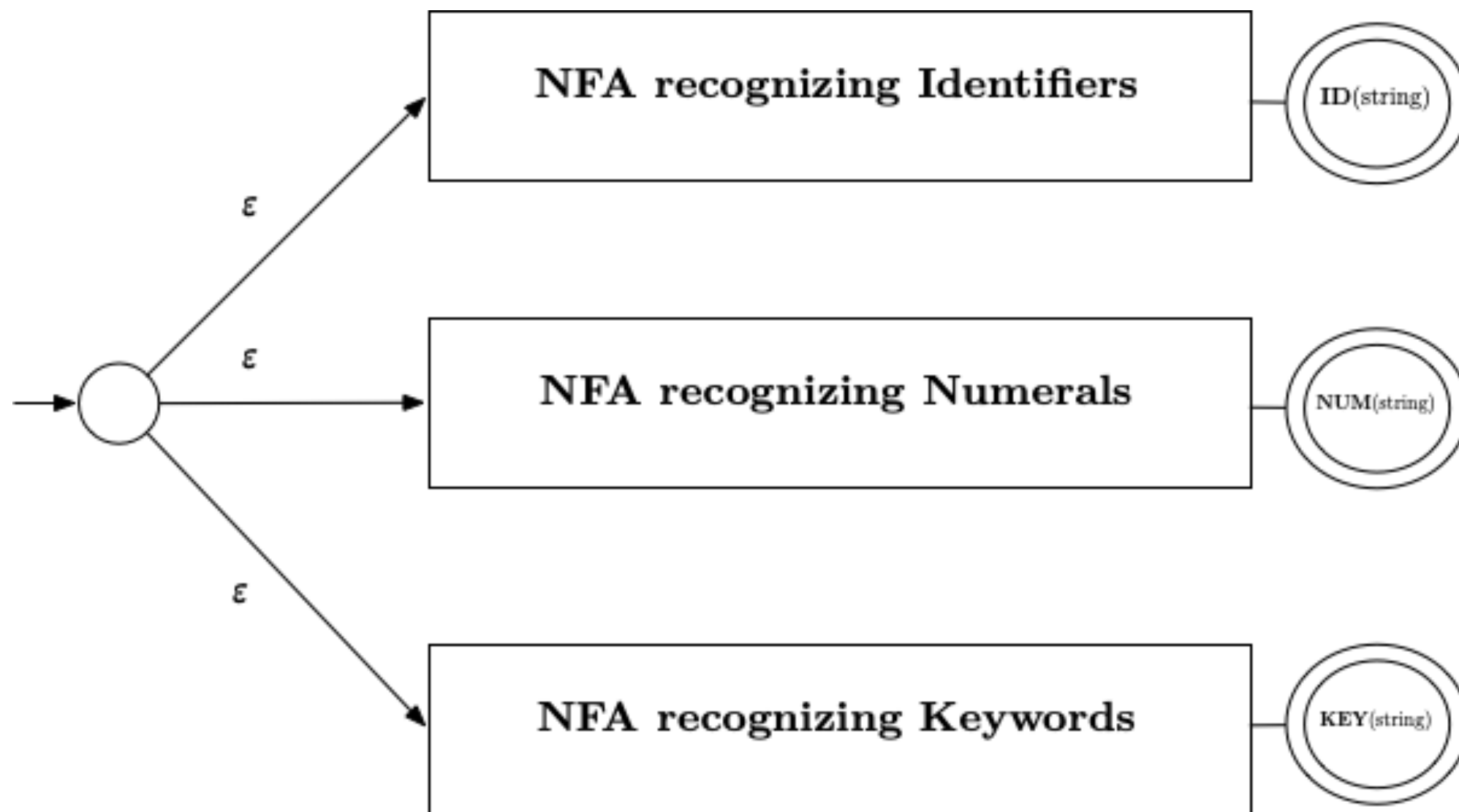
Building a lexer

We define an NFA for each class of tokens.



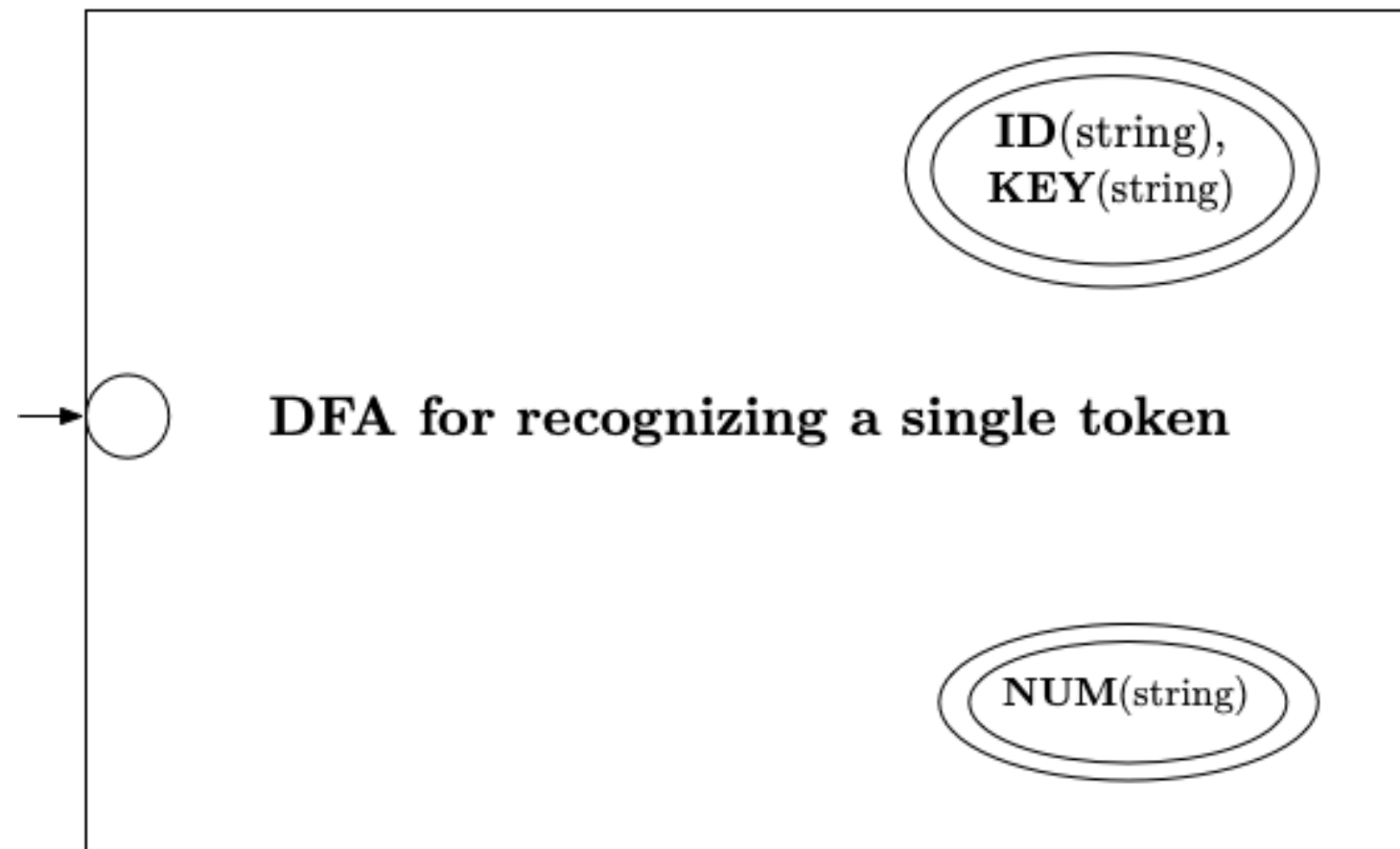
Building a lexer

Mark final states in each NFA with the token name and create an NFA that forms the union of the NFAs.



Building a lexer

Construct a small DFA using subset construction and minimization.



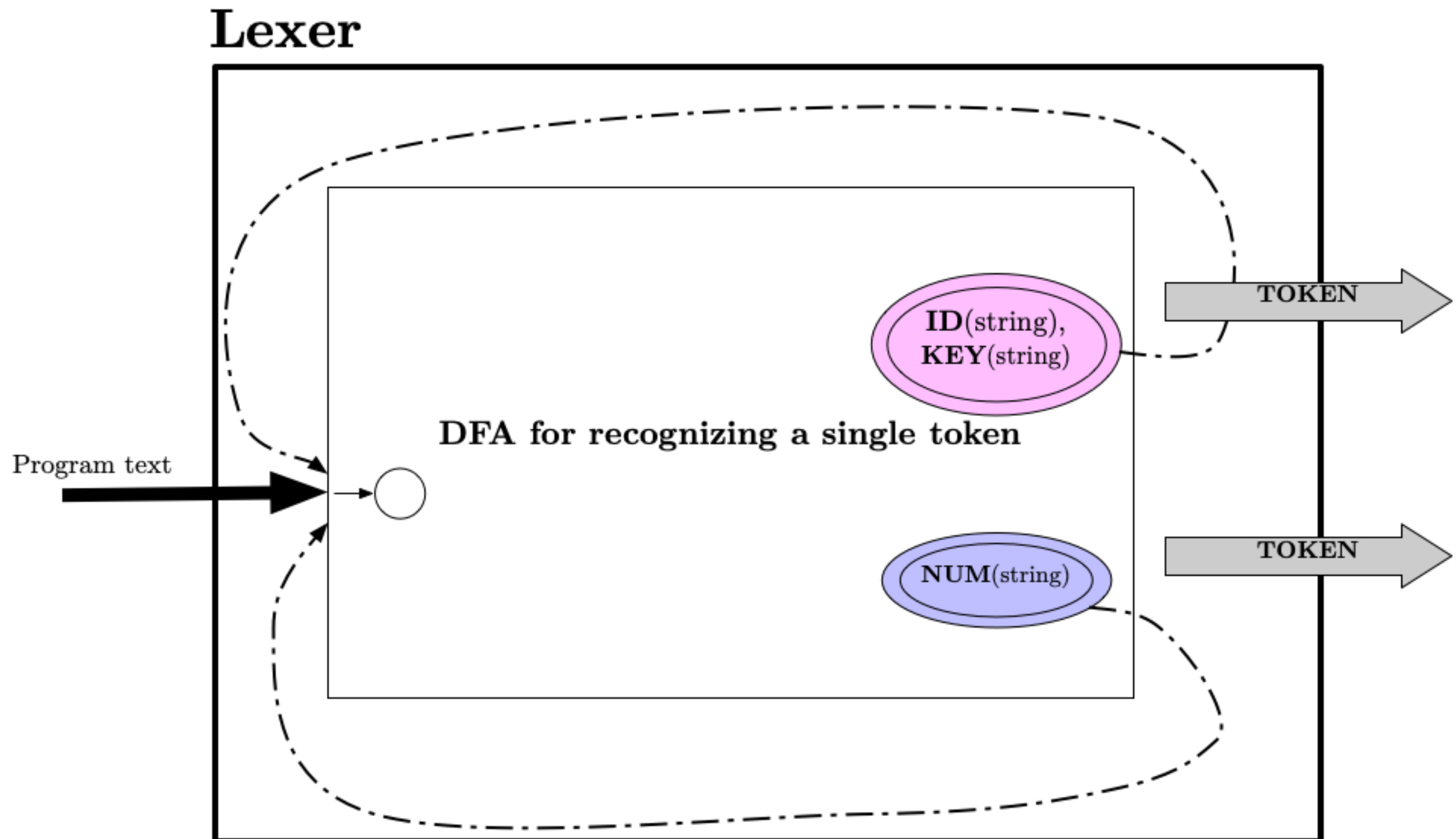
Prioritize token classes in the final DFA states so we know what the DFA is supposed to see.

Using the lexer on an input

The lexer uses a buffer to store the characters that have been read since it last saw a token.

- ① Start the DFA in normal mode,
- ② When a final state is reached, save it and continue reading
- ③ In read-ahead mode:
 - buffer all input when reading,
 - when reaching a new final state clear the buffer,
 - **if the end of input is reached** or the DFA was stuck **then** output the last final state and restore input from buffer.
- ④ Restart in normal mode until inputs ends.

Using the lexer on an input



Regular languages have limited expressiveness

A language that can be described by a regular expression is called a **regular language**. Not all languages are regular.

A simple example of such a non-regular language is the "declaration before use"-language

$$L_{DBU} = \{w\#w \mid w \in \{a,b\}^*\}$$

The *Pumping Lemma for regular languages* which is covered in the *Syntax and semantics* course can be used to prove that L_{DBU} is not regular.